

Designing Memory for Claude Code Fleets

A practical architecture guide for single-user, multi-session, and multi-agent memory

The Problem

Claude Code's memory is powerful for a single session. But the moment you run multiple sessions — a fleet of agents working on a hedge fund, a team of reviewers across repos, parallel pipelines — questions arise:

- Which agent writes to memory? Do they step on each other?
- What goes in global memory vs project memory vs nowhere?
- How does a morning-intel agent share findings with a war-daily agent?
- If 5 agents run simultaneously, who owns the MEMORY.md index?
- What memory is worth keeping vs what's ephemeral noise?

This document proposes a layered architecture for memory that scales from solo use to full agent fleets.

1. Memory Hierarchy (Inheritance Model)

Claude Code's memory loads in layers. Each layer inherits from the one above:

Layer 0: CLAUDE.md (rules, conventions, guardrails) Read-only for agents. Set by human.	← Always loaded
Layer 1: Global Memory (~/.claude/projects/~/....) User profile, cross-project patterns, feedback	← Loads for ALL sessions
Layer 2: Project Memory (project-specific path) Portfolio state, theses, pipeline configs, team ctx	← Loads when cd'd into project
Layer 3: Session Context (tasks, plans, conversation) Ephemeral. Dies on exit. Not memory.	← Current session only

What Goes Where

Layer	What belongs here	What does NOT belong
CLAUDE.md	Project conventions, tool configs, coding style, build commands	Anything that changes weekly
Global Memory	User identity, machine info, cross-project feedback, project map	Project-specific details
Project Memory	Portfolio positions, active theses, pipeline architecture, dashboards	One-off debugging notes, git-derivable info
Session Context	Current task breakdown, in-progress plans, temp state	Anything needed tomorrow

The 24-Hour Rule

If you'll need this information in a different session more than 24 hours from now, it's memory.
Otherwise, it's session context.

2. Memory File Taxonomy

Not all memories are equal. Different types have different lifespans and audiences.

By Volatility

Stable (months)	User profile, feedback, reference pointers → Layer 1 (Global)
Semi-stable (weeks)	Theses, pipeline configs, data access guides → Layer 2 (Project)
Active (days)	Portfolio positions, today's plan, sprint status → Layer 2 (Project), dated filenames
Volatile (hours)	Morning intel results, pipeline outputs → NOT memory. Use files in project dir.

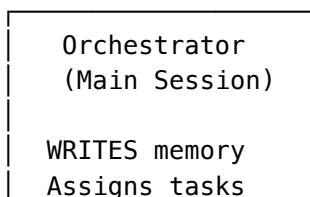
By Audience (Fleet-Critical)

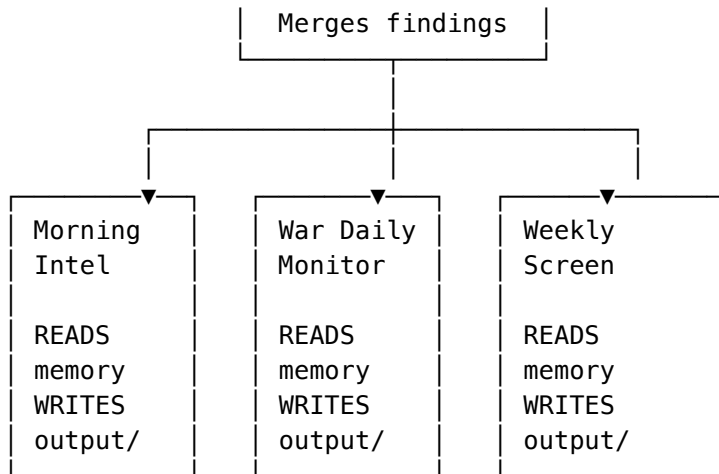
Audience	Storage	Example
All agents	MEMORY.md index (always loaded)	Portfolio snapshot, critical thresholds
Specific agents	Named memory files (loaded on demand)	meridian-pipeline.md — only Meridian agent needs this
Human only	Memory files with descriptive names	india_fund_allocation.md — human reviews, agents reference if asked
Nobody	Don't save	Debugging artifacts, one-off calculations

3. Fleet Architecture Patterns

Pattern A: Hub-and-Spoke (Recommended for Trading)

One “orchestrator” session manages memory. Worker agents read but don't write.





How it works: 1. Worker agents READ from project memory (portfolio, thresholds, theses) 2. Worker agents WRITE outputs to project directories (not memory) 3. Orchestrator reviews outputs and updates memory if anything changed 4. No agent-to-agent memory writes — eliminates conflicts

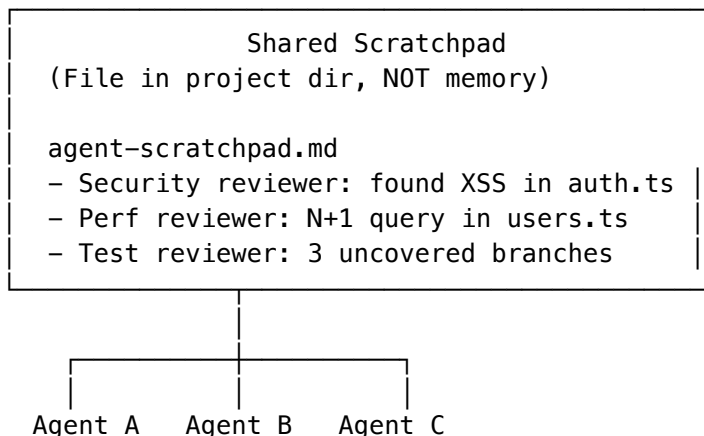
Implementation:

```

AxionHedgeFund/
├── memory/                ← Only orchestrator writes here
│   └── (MEMORY.md, etc.)
├── output/                ← Agents write here (not memory)
│   ├── morning-intel/
│   │   └── 2026-03-16.json
│   ├── war-daily/
│   │   └── 2026-03-16-12pm.json
│   └── weekly-screen/
│       └── 2026-03-16.json
├── shared/                ← Read-only shared context for all agents
│   ├── thresholds.json    ← VIX>35, OAS>400, Oil>$110
│   ├── positions.json     ← Current portfolio snapshot
│   └── watchlist.json     ← Tickers all agents should track
  
```

Pattern B: Shared Scratchpad (Agent Teams)

For Claude Code Agent Teams where teammates collaborate on a shared task.



(security) (perf) (tests)

Key insight: Agent Teams already have a mailbox/task system for communication. Don't duplicate it with memory. Use memory only for information that outlives the team session.

Pattern C: Federated Memory (Multi-Project)

When one user operates across many projects (your setup).

Global Memory (~/)

- ├ User profile (loads everywhere)
- ├ Project map (which project is where)
- ├ Cross-project feedback (coding patterns)
- ├ AxionHedgeFund Memory
 - ├ Portfolio positions
 - ├ Trading theses
 - ├ Pipeline configs
 - └ India allocation
- ├ Avaantage-BusinessApps Memory
 - ├ Brand guidelines
 - ├ PDF build pipeline
 - └ Research reports
- ├ Spirituality Memory
 - └ Cross-correlation plan
- └ (Other projects inherit only Global)

Rule: A session in ~/Axion/AxionHedgeFund/ sees Global + HedgeFund memory. It never sees Avaantage or Spirituality memory. This is by design — isolation prevents contamination.

Cross-project sharing: If something genuinely applies to all projects (like “prefer Bun over npm”), put it in Global memory or CLAUDE.md.

4. Fleet Memory Protocols

Protocol 1: Read-Many, Write-One

Only one agent writes to any given memory file. Others read.

Memory file	Writer	Readers
MEMORY.md	Orchestrator only	All agents
iran-crisis-trading.md	Research session	Trading agents
portfolio-positions.json	Portfolio updater	All trading agents
morning-intel/2026-03-16.json	Morning intel agent	War daily, orchestrator

Protocol 2: Output Directories, Not Memory

Agents produce outputs — don't store them in memory.

```

# BAD: Agent saves pipeline results to memory
---
name: Morning Intel Mar 16
type: project
---
BNO up 2.3%, VIX at 23.5, FOMC tomorrow...

# GOOD: Agent saves to output directory
# ~/Axion/AxionHedgeFund/output/morning-intel/2026-03-16.json
{
  "timestamp": "2026-03-16T06:00:00",
  "portfolio_snapshot": {...},
  "alerts": [{"ticker": "VIX", "level": 23.5, "threshold": 35}],
  "news_summary": "..."}

```

Memory is for **context that shapes future behavior**. Outputs are for **data that was produced**.

Protocol 3: Memory Promotion

Information flows upward through a promotion pipeline:

```

Session output → "VIX spiked to 42, portfolio down 8%"
      |
      ▼ (human or orchestrator decides: is this worth remembering?)
Project memory → "VIX regime change detected Mar 16. Shifted to defensive."
      |
      ▼ (only if cross-project relevant)
Global memory → (rarely promoted this high)

```

Protocol 4: Memory Expiry Tags

For fleet-generated memories that have a shelf life:

```

---
name: Monday March 16 Trading Plan
description: UVIX re-entry, FOMC positioning, India T1 deploy
type: project
expires: 2026-03-17
---

```

The expires field isn't enforced by Claude Code — it's a signal to the orchestrator or human to clean up. Agents should check dates and ignore stale memories.

Protocol 5: Conflict Resolution

When two agents could potentially update the same memory:

1. **Last write wins** — Claude Code has no locking. Accept this.
 2. **Mitigate by scoping** — give each agent its own memory files.
 3. **Mitigate by timing** — schedule agents sequentially for memory-writing tasks.
 4. **Detect conflicts** — use git (the memory dir is gitignored by default, but you can track it).
-

5. Designing for Your Hedge Fund Fleet

Current State (What You Have)

Global Memory: 5 files

- └ User profile, project map, patterns
- └ Home directory analysis

AxionHedgeFund Memory: 18 files

- └ Portfolio positions (volatile – changes daily)
- └ Trading theses (semi-stable – changes weekly)
- └ Pipeline configs (stable – changes monthly)
- └ Sprint status (active – changes in days)
- └ India allocation (semi-stable)

Proposed Fleet Architecture

Orchestrator (your main interactive session)

- └ Morning Intel Agent (6 AM ET, /loop or cron)
 - └ READS: positions.json, thresholds.json, watchlist.json
 - └ RUNS: trade morning-intel
 - └ WRITES: output/morning-intel/YYYY-MM-DD.json
- └ War Daily Agent (12 PM / 4 PM / 9 PM)
 - └ READS: positions.json, thresholds.json, iran-crisis-trading.md
 - └ RUNS: trade war-daily
 - └ WRITES: output/war-daily/YYYY-MM-DD-HHmm.json
- └ Weekly Screen Agent (Sunday evening)
 - └ READS: theses, sector watchlists
 - └ RUNS: trade weekly-screen
 - └ WRITES: output/weekly-screen/YYYY-MM-DD.json
- └ Manipulation Monitor Agent (Saturday)
 - └ READS: metals-thesis, COMEX baselines
 - └ RUNS: trade manipulation-monitor
 - └ WRITES: output/manipulation/YYYY-MM-DD.json
- └ Research Agent (on-demand)
 - └ READS: all theses, all outputs
 - └ Produces: updated thesis files, new memory entries
 - └ WRITES: Promoted to memory by orchestrator

Shared Context Files (Not Memory)

Create a shared/ directory in the project for machine-readable state:

```
// shared/positions.json – source of truth for all agents
{
  "updated": "2026-03-16T09:30:00",
  "updated_by": "orchestrator",
```

```

"positions": {
  "BNO": {"qty": 743, "avg_cost": 48.98, "current": null},
  "UCO": {"qty": 218, "avg_cost": 40.80, "current": null},
  "VIXY": {"qty": 30},
  "FAZ": {"qty": 10},
  "HUC.TO": {"qty": 40.81, "currency": "CAD"},
  "cash_usd": 21000,
  "cash_cad": 0
}
}

// shared/thresholds.json – alert triggers for all agents
{
  "VIX": {"warn": 25, "critical": 35},
  "OAS": {"warn": 350, "critical": 400},
  "OIL_WTI": {"warn": 100, "critical": 110},
  "INR_USD": {"warn": 90, "critical": 93},
  "DXY": {"warn": 108, "critical": 112}
}

// shared/watchlist.json – tickers all agents track
{
  "core": ["BNO", "UCO", "VIXY", "FAZ", "UVIX", "HUC.TO"],
  "oil": ["CL=F", "BZ=F", "STNG", "EQNR", "FRO", "TEN"],
  "defense": ["LMT", "AVAV", "RTX", "NOC"],
  "metals": ["GLD", "SLV", "GOLDBEES.NSE", "SILVERBEES.NSE"],
  "fertilizer": ["CF", "MOS", "NTR", "IPI"],
  "india": ["ONGC.NSE", "HINDPETRO.NSE", "BPCL.NSE"]
}

```

Memory File Ownership Map

File	Owner (Writer)	Frequency	Readers
MEMORY.md	Orchestrator	Daily	All
iran-crisis-trading.md	Research agent / Human	Weekly	War daily, morning intel
commodities-thesis-mar14.md	Research agent	Weekly	Weekly screen, morning intel
metals-thesis-mar14.md	Research agent	Weekly	Manipulation monitor
monday-march16-plan.md	Human / Orchestrator	Daily	All (but only on that day)
india_fund_allocation.md	Human	Monthly	India-specific agents
daily-pipeline-architecture.md	Human	Monthly	Pipeline agents (config)
data-access-guidelines.md	Human	Rarely	All (feedback type)
shared/positions.json	Orchestrator	After each trade	All agents

File	Owner (Writer)	Frequency	Readers
shared/thresholds.json	Blaise Harrison	Weekly	All agents
shared/watchlist.json	Blaise Harrison	Weekly	All agents

6. MEMORY.md Index Design

The MEMORY.md index is the bottleneck — it's always loaded, capped at 200 lines. Design it for scanability.

Anti-Pattern: Dumping Everything

```
# BAD — MEMORY.md as a database
## Portfolio
BNO 743 × $48.98 = $36,392, UCO 218 × ~$40.80...
## Research
(1) $704B bank PM derivatives (OCC) — JPM $437B...
(2) Gold decoupled from real yields...
## Pipeline
Morning intel runs at 6 AM...
```

This eats your 200-line budget on data that should be in files.

Recommended: Thin Index, Fat Files

```
# Memory — AxionHedgeFund
```

Loads for all sessions in ~/Axion/AxionHedgeFund/.

```
## Quick Reference
- **Positions:** See `shared/positions.json` (updated after each trade)
- **Thresholds:** See `shared/thresholds.json`
- **Watchlist:** See `shared/watchlist.json`
- **Today's plan:** See `monday-march16-plan.md`
```

Memory Files

Strategy & Theses

- [iran-crisis-trading.md] — War instruments, timing, rebalancing
- [commodities-thesis-mar14.md] — Oil, gas, wheat, fertilizer
- [metals-thesis-mar14.md] — Gold, silver, copper, rare earths
- [stagflation-scenarios-mar15.md] — 4 scenarios with indicators

Configuration

- [daily-pipeline-architecture.md] — 4 automated pipelines
- [data-access-guidelines.md] — API vs MCP vs Grok decision tree

India

- [india_fund_allocation.md] — Rs 77L breakdown, Veena Nigam

Sub-Projects

- [meridian-pipeline.md] — Quant platform architecture
- [avaantage-details.md] — 488 diligence reports

Result: ~30 lines instead of 85. Agents know where to look. Details are in files.

7. Fleet-Specific Memory Patterns

Pattern: Shift Handoff

When agents run at different times (morning ☐ midday ☐ evening), each should leave a handoff note:

```
output/handoff/
├─ morning-to-midday.md    ← "VIX elevated, watch BNO. No alerts triggered."
├─ midday-to-evening.md    ← "Oil spiked on Hormuz news. VIXY up 12%."
└─ evening-to-morning.md  ← "After-hours: futures flat. No action needed."
```

The next agent reads the previous handoff as part of its startup context.

Pattern: Alert Escalation

Agents should escalate critical findings to a shared location:

```
output/alerts/
├─ 2026-03-16-vix-spike.md
│   priority: critical
│   message: "VIX crossed 35 threshold at 2:15 PM"
│   action_required: true
│   acknowledged: false
└─ 2026-03-16-oil-drop.md
    priority: warning
    message: "WTI dropped below $95 support"
    action_required: false
```

The orchestrator (or a /loop monitor) scans this directory and surfaces unacknowledged alerts.

Pattern: Consensus Building

When multiple research agents analyze the same topic:

```
output/consensus/
├─ oil-outlook/
│   ├── agent-macro.md    ← "Bearish: demand destruction signal"
│   ├── agent-supply.md   ← "Bullish: Hormuz disruption ongoing"
│   ├── agent-technical.md ← "Neutral: testing support at $95"
│   └─ synthesis.md       ← Orchestrator combines: "Hold. Physical disruption
│                           outweighs demand signal. Watch $92 support."
```

The synthesis gets promoted to memory if it changes the thesis.

Pattern: Memory Snapshots

Before major events (FOMC, earnings, geopolitical), snapshot memory state:

```
snapshots/
├─ pre-fomc-2026-03-19/
│   ├── positions.json
│   └─ thresholds.json
```

```

├── thesis-summary.md
├── post-fomc-2026-03-19/
│   ├── positions.json
│   └── what-changed.md

```

This gives you a diff of how your thinking evolved around key events.

8. Implementation Checklist

Phase 1: Immediate (Today)

- ☐ Create shared/ directory with positions.json, thresholds.json, watchlist.json
- ☐ Create output/ directory structure (morning-intel, war-daily, weekly-screen, manipulation)
- ☐ Slim down MEMORY.md to thin index format (30 lines, not 85)
- ☐ Move volatile data (daily positions, research bullet points) out of MEMORY.md into files

Phase 2: This Week

- ☐ Set up /loop or cron for morning-intel and war-daily agents
- ☐ Define agent SKILL.md files that specify which shared/ files to read
- ☐ Create handoff note convention in output/
- ☐ Test: two parallel sessions, verify no memory write conflicts

Phase 3: This Month

- ☐ Add alert escalation directory and /loop monitor
 - ☐ Build consensus pattern for weekend research sessions
 - ☐ Implement memory snapshots before FOMC/major events
 - ☐ Evaluate: which memory files are actually being read? Prune unused ones
 - ☐ Consider autoMemoryDirectory setting to centralize all project memories
-

9. Memory Anti-Patterns

Anti-Pattern	Why It's Bad	Do This Instead
Every agent writes MEMORY.md	Race conditions, index bloat	Single writer (orchestrator)
Pipeline outputs in memory	Memory is for context, not data	Write to output/ directory
Positions in MEMORY.md body	Changes daily, wastes 200-line budget	shared/positions.json
Duplicate across layers	Global + project both have portfolio	One canonical location
Undated memories	Can't tell if still relevant	Include dates, use expires
Memory as todo list	Tasks tool exists for this	Use Tasks for current session
Agent-specific temp state	Pollutes shared memory	Agent writes to its own output dir
Saving git-derivable info	Redundant, goes stale	git log / git blame

10. Decision Framework

When you're about to save something to memory, run through this:

Is this derivable from code, git, or project files?

- YES: Don't save. Read the source.
- NO: Continue.

Will I need this in a future session (>24 hours)?

- NO: Use session context (Tasks/Plans).
- YES: Continue.

Is this specific to one project?

- YES: Project memory.
- NO: Global memory.

Does this change daily?

- YES: shared/ directory (JSON), not memory.
- NO: Memory file with frontmatter.

Should all agents see this immediately?

- YES: MEMORY.md index or shared/ directory.
- NO: Named memory file (loaded on demand).

Am I the designated writer for this memory?

- YES: Write it.
- NO: Write to output/ and let the owner promote it.

Summary

Principle	Implementation
Hierarchy	CLAUDE.md □ Global Memory □ Project Memory □ Session Context
Fleet protocol	Read-many, write-one. Orchestrator owns MEMORY.md.
Separation	Memory = context. Output = data. Shared = live state.
Index discipline	MEMORY.md stays under 40 lines. Fat files, thin index.
Conflict avoidance	File ownership map. No two agents write the same file.
Handoffs	Agents leave notes for the next shift in output/handoff/.
Promotion	Outputs □ Memory only when they change future behavior.
Expiry	Date everything. Prune weekly. Snapshot before events.